

JAVA LAB RECORD

Student Performance Analysis System

1. Basic Information

Title	Student Performance Analysis System
Real-time Application	College Student Performance Analysis
Programming Language	Java
Main Class	StudentPerformance
Package	Excercise

2. Objective / Aim

To design and implement a student performance analysis system in Java using Object-Oriented Programming principles to calculate marks, percentage, grade, highest and lowest marks, and overall academic performance.

3. System Requirements

Hardware	Standard PC setup; Core i3/i5 processor; 4GB/8GB RAM
Operating System	Windows / Linux / macOS
JDK	JDK 17 or JDK 21
IDE / Editor	Eclipse / IntelliJ IDEA / NetBeans / VS Code

4. Problem Statement

Develop a Java application for a college to analyze the academic performance of a student. The system accepts student details and marks, stores five subject marks in an array, calculates total and average marks, finds the highest and lowest marks, calculates and rounds percentage, assigns a grade, determines pass/fail status, formats the student's name using String operations, and displays a performance remark.

5. Algorithm / Logic Description

1. Create a class named Student.
2. Declare roll number, student name, and an integer array for five subject marks.
3. Create a constructor to initialize the student details.
4. Create methods to calculate total, average, highest mark, lowest mark, percentage, grade, result, and performance remark.
5. Use a loop to calculate the total marks.
6. Use Math.max() and Math.min() to determine the highest and lowest marks.
7. Calculate percentage using the maximum total of 500 marks.
8. Use Math.round() to round the percentage to two decimal places.
9. Use trim(), toUpperCase(), and length() to process the student's name.
10. Assign grades according to the specified percentage ranges.

11. Consider the student PASS when percentage is at least 50%; otherwise display FAIL.

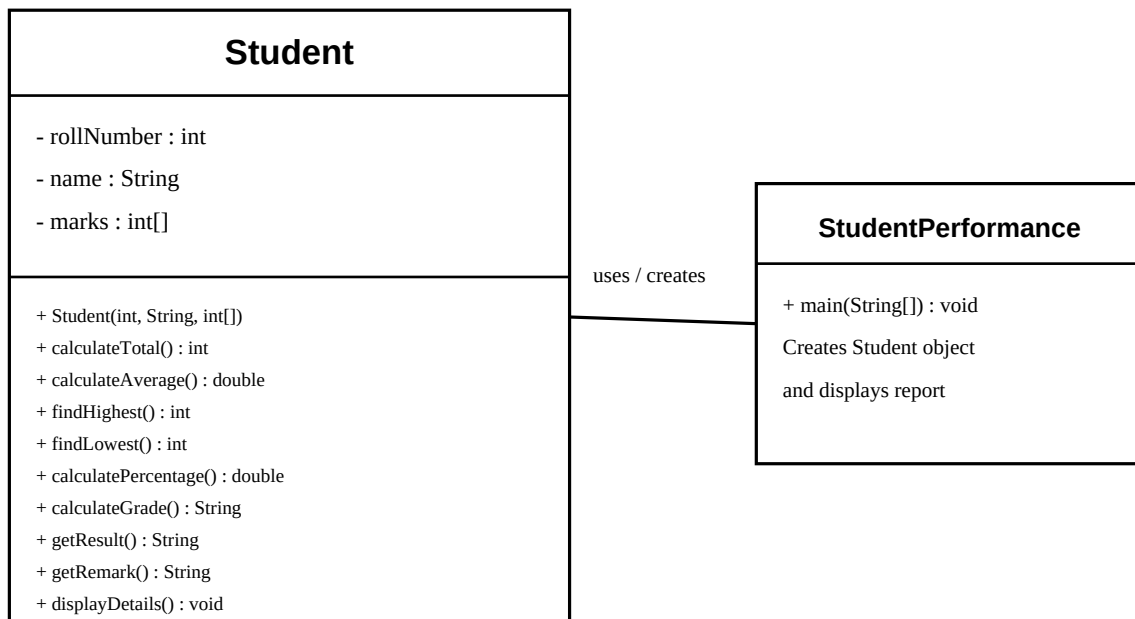
12. Create a Student object in main(), then display the complete performance report.

Grade Criteria

Percentage	Grade
90–100	A+
80–89	A
70–79	B
60–69	C
50–59	D
Below 50	F (Fail)

6. UML Class Diagram

The UML diagram below shows the Student class, its data members, methods, and the relationship with the StudentPerformance main class.



7. Source Code

The supplied code has been formatted into valid Java syntax. The visible formatting artifacts in the pasted text (such as Math.*max*, System.**out***, and escaped multiplication signs) have been normalized so the source is directly compilable.

```
package Excercise;

import java.util.Scanner;

class Student {

    int rollNumber;
    String name;
    int[] marks;

    // Constructor
    Student(int rollNumber, String name, int[] marks) {
        this.rollNumber = rollNumber;
        this.name = name;
        this.marks = marks;
    }

    // Calculate total marks
    int calculateTotal() {
        int total = 0;

        for (int mark : marks) {
            total += mark;
        }

        return total;
    }

    // Calculate average marks
    double calculateAverage() {
        return calculateTotal() / (double) marks.length;
    }

    // Find highest mark
```

```

int findHighest() {
    int highest = marks[0];

    for (int mark : marks) {
        highest = Math.max(highest, mark);
    }

    return highest;
}

// Find lowest mark
int findLowest() {
    int lowest = marks[0];

    for (int mark : marks) {
        lowest = Math.min(lowest, mark);
    }

    return lowest;
}

// Calculate percentage
double calculatePercentage() {
    // Maximum marks = 5 subjects x 100
    double percentage = (calculateTotal() / 500.0) * 100;

    // Round to two decimal places using Math.round()
    return Math.round(percentage * 100.0) / 100.0;
}

// Determine grade
String calculateGrade() {
    double percentage = calculatePercentage();

    if (percentage >= 90)
        return "A+";
    else if (percentage >= 80)
        return "A";
    else if (percentage >= 70)
        return "B";
    else if (percentage >= 60)
        return "C";
    else if (percentage >= 50)
        return "D";
    else
        return "F";
}

// Determine pass or fail
String getResult() {
    if (calculatePercentage() >= 50)
        return "PASS";
    else
        return "FAIL";
}

// Performance remark
String getRemark() {
    String grade = calculateGrade();

    switch (grade) {
        case "A+":
            return "Excellent Performance";
        case "A":
            return "Very Good Performance";
        case "B":
            return "Good Performance";
        case "C":
            return "Average Performance";
        case "D":
            return "Satisfactory Performance";
        default:
            return "Needs Improvement";
    }
}

// Display student details
void displayDetails() {

    // String operations

```

```

String formattedName = name.trim().toUpperCase();
int nameLength = formattedName.length();

System.out.println("\n===== STUDENT PERFORMANCE REPORT =====");
System.out.println("Roll Number      : " + rollNumber);
System.out.println("Student Name      : " + formattedName);
System.out.println("Name Length      : " + nameLength);

System.out.println("\nSubject Marks:");

for (int i = 0; i < marks.length; i++) {
    System.out.println("Subject " + (i + 1) + "      : " + marks[i]);
}

System.out.println("\nTotal Marks      : " + calculateTotal());
System.out.printf("Average Marks      : %.2f\n", calculateAverage());
System.out.println("Highest Mark      : " + findHighest());
System.out.println("Lowest Mark       : " + findLowest());
System.out.printf("Percentage       : %.2f%%\n", calculatePercentage());
System.out.println("Grade             : " + calculateGrade());
System.out.println("Result            : " + getResult());
System.out.println("Performance Remark: " + getRemark());

System.out.println("=====");
}
}

public class StudentPerformance {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Roll Number: ");
        int rollNumber = sc.nextInt();
        sc.nextLine();

        System.out.print("Enter Student Name: ");
        String name = sc.nextLine();

        int[] marks = new int[5];

        System.out.println("Enter marks for 5 subjects:");

        for (int i = 0; i < marks.length; i++) {
            System.out.print("Subject " + (i + 1) + ": ");
            marks[i] = sc.nextInt();
        }

        // Create Student object
        Student student = new Student(rollNumber, name, marks);

        // Display complete performance report
        student.displayDetails();

        sc.close();
    }
}

```

8. Compilation & Execution Commands

Save the source file exactly as the public class name: **StudentPerformance.java**. Because the source declares the package **Excercise**, compile and run it from the directory above the Excercise folder.

```
javac Excercise\StudentPerformance.java
java Excercise.StudentPerformance
```

If the package declaration is removed, the commands can instead be:

```
javac StudentPerformance.java
java StudentPerformance
```

9. Output Screen

Sample execution using Roll Number 101, name " Rahul Kumar ", and marks 85, 92, 78, 88, 95:

```
Command Prompt / Terminal
Enter Roll Number: 101
Enter Student Name:  Rahul Kumar
Enter marks for 5 subjects:
Subject 1: 85
Subject 2: 92
Subject 3: 78
Subject 4: 88
Subject 5: 95

===== STUDENT PERFORMANCE REPORT =====
Roll Number      : 101
Student Name     : RAHUL KUMAR
Name Length      : 11

Subject Marks:
Subject 1        : 85
Subject 2        : 92
Subject 3        : 78
Subject 4        : 88
Subject 5        : 95

Total Marks      : 438
Average Marks    : 87.60
Highest Mark     : 95
Lowest Mark      : 78
Percentage       : 87.60%
Grade            : A
Result           : PASS
Performance Remark: Very Good Performance
=====
```

10. Result

The Student Performance Analysis System was successfully implemented in Java using OOP concepts, arrays, String operations, Math methods, and built-in packages. The program successfully calculates and displays the student's academic performance, including total marks, average, highest and lowest marks, percentage, grade, result, and performance remark.

11. Conclusion

The program demonstrates how a Java class and object can be used to organize student data and performance calculations. It also demonstrates arrays, constructors, methods, loops, conditional statements, switch, String operations, Math methods, and Scanner input in a single practical application.